

REDES RECORRENTES

LISTA DE EXERCÍCIOS

Redes Recorrentes

- I) Qual é a principal vantagem das redes neurais recorrentes (RNNs) em relação a MLPs e redes convolucionais?
- II) Descreva a estrutura de uma RNN básica.
- III) Em frameworks de desenvolvimento de redes neurais (como o PyTorch) é comum que o bloco de rede recorrente conte apenas com a camada densa que computa o estado oculto (*hidden state*) e que não conte com a camada densa que computa a saída. Qual motivo disso?
- IV) Uma rede recorrente tem um limite teórico para o tamanho de sequências que consegue processar? E do ponto de vista prático, existem limites?
- V) Redes recorrentes tendem a sofrer mais dos problemas do gradiente que se dissipa e do gradiente explosivo. Por que isso acontece?
- VI) (*Adaptado de CS224N Stanford 2017*) Considere que todos os parâmetros da rede recorrente ilustrada na figura abaixo são escalares (θ_{hh} , θ_{xh} , b_h , θ_{hy} e b_y). A função de ativação $f[\cdot]$ é a função sinal definida abaixo:

$$f(z) = \begin{cases} z \geq 0, & 1, \\ z < 0, & 0 \end{cases}$$

Sua tarefa é descobrir valores para esses parâmetros que façam a rede recorrente ter como saída inicial o valor 0 e, assim que ela receba um valor 1 como entrada ela passe a ter como saída o valor 1 para toda e qualquer entrada futura. Por exemplo: Se a entrada for 00110 a saída deve ser 00111.

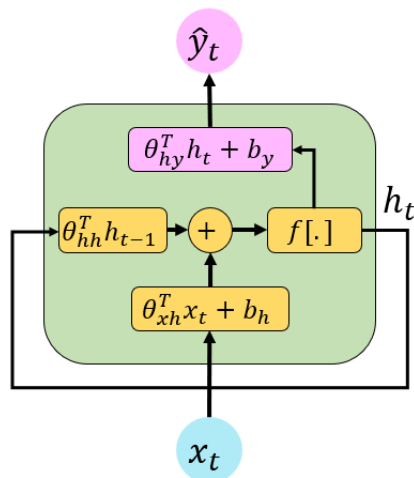


Figure 1: RNN para questão VI

VII) Considere uma rede híbrida que faz *Image Captioning* (gera legendas para imagens). A arquitetura é composta por um *encoder* (uma CNN pré-treinada, como a VGG-16) e um *decoder* (uma RNN simples). O encoder recebe como input uma imagem I e sua saída serve de entrada para o decoder, que então é responsável pela geração de legenda, uma palavra de cada vez ($y_1, y_2, \dots, y_n \dots$). Com base neste cenário, responda:

- a) Caso utilizemos uma VGG-16, qual é o papel dela nesta tarefa? E qual é o papel da RNN? Por que não podemos usar apenas a VGG-16 para a tarefa inteira?
- b) Este processo de geração de palavras é autorregressivo. Descreva o que serve como input (x_t) e o que serve como estado oculto anterior (h_{t-1}) para a RNN nos seguintes passos de tempo:
 - i.) Para gerar a primeira palavra (y_1) no tempo $t = 1$.
 - ii.) Para gerar a segunda palavra (y_2) no tempo $t = 2$.
- c) A CNN (Encoder) processa a imagem uma única vez para gerar um vetor de contexto v . Qual é a origem e o formato (shape) mais comum deste vetor v ? Ele é a saída da última camada convolucional (um tensor 3D), ou é a saída que foi "achatada" (flattened), ou é a saída já processada de uma camada totalmente conectada (FC)?
- d) Digamos que não queremos perder a informação espacial de cada frame ao passá-la para a RNN. Ou seja, queremos que o *input* x_t e o *estado oculto* h_t sejam eles mesmos mapas de características 3D (ex: [Altura, Largura, Canais]), e não vetores 1D. É possível utilizar operações de convolução *dentro* da célula da RNN (para calcular h_t a partir de h_{t-1} e x_t) em vez das multiplicações de matrizes (camadas densas) tradicionais? Explique brevemente o conceito.

Gabarito

- I) As RNNs podem capturar e analisar informações sequenciais, tornando-as adequadas para tarefas que envolvem séries temporais, processamento de linguagem natural e dados sequenciais. Elas atingem esse feito usando uma recorrência, ou seja, um estado oculto da rede é computado e serve como entrada para o processamento do próximo item da sequência.
- II) Uma RNN consiste em uma camada de entrada, uma camada oculta com conexões recorrentes e uma camada de saída. A camada oculta mantém um estado oculto que é atualizado a cada passo temporal, e é usado como entrada para os passos subsequentes, permitindo que a rede memorize informações de entradas anteriores.
- III) As tarefas que envolvem dados sequenciais são muito variadas. Podemos fazer análise de sentimentos (classificar uma sequência), *image captioning* no qual temos uma entrada e uma sequência como saída ou ainda *sequence-to-sequence* onde temos sequências na entrada e na saída. Dada essa grande variedade de problemas, seria um desperdício manter parâmetros de saída para cada unidade recorrente. Mantendo a unidade recorrente sem saída por padrão, os *frameworks* transferem a responsabilidade de definir quando a saída deve ser computada (e onde os parâmetros devem ser aprendidos) para o desenvolvedor.
- IV) Do ponto de vista teórico uma RNN consegue processar entradas de qualquer tamanho, um item por vez. Contudo, na prática vemos que RNN não trabalham bem com dependências muito longas. Isso ocorre pois em RNN estamos sempre atualizando o estado oculto em cada uma das iterações ao longo do tempo.
- V) Redes recorrentes são mais sensíveis a esse tipo de problema pois além de serem profundas no número de camadas elas são profundas também na dimensão tempo. Dessa forma, durante o *backpropagation through time* o gradiente sofre mais multiplicações e é acumulado em múltiplas iterações potencializando ambos problemas.
- VI) Existem muitas soluções para essa questão. A solução abaixo é apenas uma das possibilidades:

$$\begin{aligned}\theta_{hh} &= 1 \\ \theta_{xh} &= 1 \\ b_h &= -1 \\ \theta_{hy} &= 1 \\ b_y &= 0\end{aligned}$$
- VII)
 - a) Sua função é processar a imagem, analisando e extraíndo as características visuais importantes, comprimindo toda a informação em um único vetor (v). Usando uma VGG-16 de encoder (como a VGG) não é possível, sozinha, gerar uma saída sequencial. A tarefa de decoder sequencial (gerar uma frase) requer uma arquitetura diferente, como uma RNN ou um Transformer.
 - b) O processo é chamado de autorregressivo porque a saída de um passo de tempo é usada como entrada para o próximo passo.
 - i. **Para gerar y_1 (Tempo $t = 1$):**
 - **Estado Oculto Anterior** (h_{t-1}): É o h_0 , que é o vetor da imagem v fornecido pela CNN.
 - **Input** (x_1): Não temos uma palavra anterior. Portanto, usamos um *token* especial de início de sentença (ex: <START>).

- **Cálculo:** $h_1 = f(W_{hh} \cdot v + W_{xh} \cdot x_1 + b_h)$ e $y_1 = g(W_{hy} \cdot h_1 + b_y)$.
- ii. **Para gerar y_2 (Tempo $t = 2$):**
 - **Estado Oculto Anterior** (h_{t-1}): É o h_1 , o estado oculto que acabou de ser calculado no passo anterior.
 - **Input** (x_2): É a palavra que foi gerada no passo anterior, ou seja, y_1 .
 - **Cálculo** $h_2 = f(W_{hh} \cdot h_1 + W_{xh} \cdot y_1 + b_h)$ e $y_2 = g(W_{hy} \cdot h_2 + b_y)$.
- c) A abordagem mais comum e direta é usar a saída ‘achatada’ (flattened) da última camada convolucional.
 - i. Processo: A imagem passa pelas camadas convolucionais (ex: VGG-16), resultando em um *feature map* 3D (ex: $7 \times 7 \times 512$). Achatamos (flattening) para um vetor 1D (ex: 25088×1). A saída é usada como o vetor v .
 - ii. Formato (Shape): O vetor v é um vetor 1D (ex: ‘(25088,)’).
 - iii. Uso: Este vetor 1D representa a imagem inteira, é usado para inicializar o primeiro estado oculto (h_0) da RNN, ‘preparando’ o decoder com o que ele deve descrever.
- d) Sim, é possível, e esta arquitetura é conhecida como **Convolutional RNN** (ou ConvLSTM/ConvGRU).
 - i. Conceito: Em uma RNN padrão, o cálculo do novo estado oculto envolve multiplicações de matrizes (camadas densas):

$$h_t = \tanh(W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t + b_h)$$

Exigindo que h_t , h_{t-1} e x_t sejam vetores 1D.

- ii. Em uma ConvRNN: As multiplicações de matrizes $W_{hh} \cdot$ e $W_{xh} \cdot$ são substituídas por operações de convolução. Neste caso, o *input* x_t é o próprio mapa de características 3D da CNN (ex: $7 \times 7 \times 512$). É necessário que sejam preservadas as dimensões espaciais, permitindo que a RNN “lembre” não apenas o que viu nos frames anteriores, mas onde (espacialmente) viu.